Scalability.md

Avora — Complete Architecture & Scalability Audit

Objective

Perform a complete production architecture, performance, reliability, and scalability audit of the Avora codebase.

The primary question is:

Can Avora reliably scale from its current state to 1,000–5,000 active student users without architectural failure, unacceptable latency, database instability, excessive infrastructure costs, or degraded user experience?

Do not assume that the current architecture is scalable. Verify it from the actual codebase, configuration, database schema, deployment configuration, tests, and infrastructure.

This is an audit first, remediation second task.

Do not make speculative architectural changes before establishing evidence for the problem.

———

1. Audit Rules

You MUST:

1. Inspect the entire repository before reaching conclusions.

2. Understand the existing architecture and data flow.

3. Identify critical paths used by:

    * Students

    * Tutors

    * Parents

    * AI features

    * Authentication

    * Homework/submissions

    * Exam marking

    * Readiness calculations

    * Dashboards

    * Notifications/background processing

4. Trace requests from:

    Frontend → API → services → database/external APIs → response

5. Inspect database access patterns and identify expensive queries.

6. Inspect asynchronous/concurrent operations.

7. Inspect external API dependencies and their rate limits.

8. Inspect deployment and infrastructure configuration.

9. Inspect existing tests and determine what they actually prove.

10. Use available ECC agents, skills, hooks, subagents, and other repository-analysis capabilities wherever they improve the audit.

11. Use multiple independent agents for important areas rather than relying on a single analysis.

12. Look for interactions between bottlenecks, not just isolated problems.

———

2. Do NOT Immediately Fix Everything

The first phase must be an audit-only phase.

Do not refactor large sections of the application simply because a pattern could theoretically be improved.

For every identified issue, establish:

* Evidence
* Location
* Severity
* Why it becomes a problem at scale
* Estimated impact
* Whether it is currently blocking
* Whether it is likely to become a problem at 1k users
* Whether it is likely to become a problem at 5k users
* Recommended remediation
* Priority

———

3. Architecture Mapping

Create a complete architecture map.

Document:

Frontend

* Application structure
* State management
* API communication
* Query caching
* Rendering patterns
* Large data sets/tables
* Network request patterns
* Duplicate requests
* Polling
* File uploads
* Error handling

Backend

* FastAPI application structure
* Routers
* Services
* Business logic
* Dependencies
* Middleware
* Authentication
* Authorization
* Async architecture
* Background processing
* External API calls

Database

* PostgreSQL schema
* Tables
* Relationships
* Foreign keys
* Indexes
* Constraints
* Query patterns
* Transactions
* Connection pooling
* N+1 queries
* Full-table scans
* Large joins
* Pagination
* Sorting/filtering
* Data growth patterns

AI infrastructure

Audit every AI-dependent workflow.

Determine:

* Which operations call AI APIs
* Which are synchronous
* Which are asynchronous

* Which block HTTP requests

* Token usage

* Expected latency

* Retry behavior

* Rate-limit behavior

* Failure behavior

* Concurrent request behavior

* Cost scaling

* Queue requirements

* Idempotency

* Duplicate requests

* Caching opportunities

Infrastructure

Inspect:

* Docker

* Render/deployment configuration

* Environment configuration

* CPU

* Memory

* Database resources

* Workers/processes

* Autoscaling

* Storage

* Networking

* Logs

* Monitoring

* Health checks

* Timeouts

* Retry policies

---

4. Scalability Model

Build a realistic workload model for:

100 active students

500 active students

1,000 active students

2,500 active students

5,000 active students

Do NOT equate active users with concurrent users.

Model realistic behavior.

For example, estimate:

* Requests per active student per minute

* Peak requests/second

* Dashboard loads

* Homework submissions

* AI requests

* Exam marking requests

* File uploads

* Tutor dashboard requests

* Background jobs

* Database reads/writes

* Average payload sizes

* Peak traffic

Clearly state assumptions.

If the actual application provides evidence allowing better estimates, use the application's real behavior instead.

---

5. Identify Bottlenecks

Search aggressively for:

Database bottlenecks

* Missing indexes

* Poor indexes
* N+1 queries
* Repeated queries
* Unnecessary joins
* Full table scans
* Inefficient ORM usage
* Loading entire collections
* Missing pagination
* Inefficient aggregation
* Long transactions
* Excessive transaction scope
* Connection pool exhaustion
* Race conditions
* Lock contention
* Duplicate writes
* Unnecessary database round trips

API bottlenecks

* Blocking synchronous work
* Slow endpoints
* Excessive serialization
* Huge responses
* Repeated computation
* Missing caching
* Excessive middleware
* Poor concurrency

* Unbounded requests
* Missing rate limiting
* Inefficient dependency injection

AI bottlenecks

* Synchronous AI calls
* Excessive latency
* Rate limits
* Retry storms
* Duplicate AI calls
* Lack of queues
* Lack of concurrency control
* No timeout handling
* No fallback behavior
* Cost explosions

Frontend bottlenecks

* Excessive API requests
* Duplicate requests
* Poor caching
* Large payloads
* Large component renders
* Unnecessary rerenders
* Huge tables
* Missing virtualization

* Poor pagination
* Slow initial load

Infrastructure bottlenecks

* Single points of failure
* Insufficient workers
* Memory leaks
* CPU saturation
* Database saturation
* Storage limitations
* Missing autoscaling
* Missing health checks
* Poor deployment configuration

---

6. Reliability Audit

Scalability is not only about speed.

Determine what happens when:

* Database becomes slow
* Database temporarily disconnects
* AI provider times out
* AI provider rate-limits Avora

* A request is duplicated

* A background job fails

* A user refreshes during an operation

* A large file is uploaded

* Multiple users modify the same resource

* A worker crashes

* An external API becomes unavailable

Identify whether the system has:

* Retries

* Timeouts

* Idempotency

* Transactions

* Rollbacks

* Dead-letter handling

* Graceful degradation

* Error recovery

* Monitoring

* Alerting

———

7. Security + Multi-Tenant Scalability

Audit whether scaling users can create security or isolation problems.

Pay particular attention to:

* Tutor/student data isolation
* Parent/student permissions
* Class-level authorization
* Resource ownership
* IDOR vulnerabilities
* Cross-tenant queries
* Database filtering
* Authorization at service level
* File access
* AI context isolation

A performance optimization MUST NOT compromise data isolation.

---

8. Load Testing

Where possible, create or use realistic load tests.

Do not simply benchmark one endpoint.

Test realistic workflows such as:

Student workflow

Login → dashboard → readiness → planner → homework → AI feature

Tutor workflow

Login → class → student list → student profile → homework → readiness

AI workflow

Student request → API → AI provider → processing → database → response

Test increasing loads and record:

* Requests/second
* Average latency
* p50
* p95
* p99
* Error rate
* CPU
* Memory
* Database connections
* Database latency
* AI latency
* Queue depth where applicable

If actual load testing cannot be performed because of infrastructure limitations, explicitly state this and explain what is missing.

Do NOT fabricate benchmark results.

———

9. Bottleneck Classification

Classify every issue:

P0 — Critical

System is likely to fail or become unsafe at relatively low scale.

P1 — High

Likely to materially affect 1k–5k users.

P2 — Medium

Will become important as usage grows.

P3 — Low

Optimization opportunity but not currently important.

Also classify each issue as:

* Database

* Backend

* Frontend

* AI

* Infrastructure

* Security

* Reliability

* Architecture

* Cost

---

10. Scalability Verdict

Provide an explicit verdict.

Answer:

Can Avora currently support 100 active students?

Can Avora currently support 500 active students?

Can Avora currently support 1,000 active students?

Can Avora currently support 2,500 active students?

Can Avora currently support 5,000 active students?

For each:

🟢 Ready

🟡 Requires remediation

🔴 Not currently safe

Explain WHY.

Do not give a green rating without evidence.

———

11. Scaling Roadmap

Create a staged remediation plan:

Stage 0

Current architecture → 100 active students

Stage 1

100 → 500

Stage 2

500 → 1,000

Stage 3

1,000 → 2,500

Stage 4

2,500 → 5,000

For each stage specify:

* Required code changes
* Database changes
* Infrastructure changes
* Monitoring requirements
* Load tests
* New services if necessary
* Estimated complexity
* Dependencies
* What can remain unchanged

Do NOT recommend microservices simply because they are theoretically scalable.

Prefer the simplest architecture capable of meeting the requirement.

---

12. AI Agent Remediation Prompts

After completing the audit, create a separate section containing implementation-ready prompts for AI coding agents.

Each prompt must include:

Issue

What is wrong.

Evidence

Exact files/functions/queries involved.

Impact

Why it matters.

Target

What the agent should achieve.

Constraints

What it must NOT break.

Implementation

What should be changed.

Tests

What tests must be added/modified.

Verification

How to prove the fix works.

Performance target

Where measurable, define a target.

———

13. Agent Specialization

Use the available ECC infrastructure intelligently.

Where possible, assign independent agents/subagents to:

1. Database audit
2. Backend/API audit

3. Frontend performance audit

4. AI pipeline audit

5. Infrastructure/deployment audit

6. Security/multi-tenancy audit

7. Testing/load-testing audit

8. Architecture review

Then have a final synthesis/reviewer agent compare their findings.

Do not blindly trust individual agent conclusions.

Look for conflicting findings and resolve them using evidence from the repository.

---

14. Required Final Deliverable

Produce a detailed document containing:

1. Executive summary

2. Current architecture

3. Architecture diagram/description

4. Scalability assumptions

5. Workload model

6. Database audit

7. Backend audit

8. Frontend audit

9. AI pipeline audit

10. Infrastructure audit

11. Reliability audit

12. Security/multi-tenancy audit

13. Load-testing results

14. Bottleneck inventory

15. P0/P1/P2/P3 classification

16. 100-user assessment

17. 500-user assessment

18. 1k-user assessment

19. 2.5k-user assessment

20. 5k-user assessment

21. Cost/scaling considerations

22. Remediation roadmap

23. AI implementation prompts

24. Required tests

25. Final production-readiness verdict

---

15. Critical Instruction

Do not optimize for producing a long report. Optimize for discovering real bottlenecks.

If you find no evidence of a problem, say so.

If the current architecture is sufficient, say so.

If a component does not need to be changed, explicitly say:

NO CHANGE REQUIRED

Do not introduce unnecessary infrastructure, microservices, caching, queues, or abstractions merely because they are common scalability patterns.

The goal is not to make Avora theoretically capable of supporting millions of users.

The goal is to determine:

What prevents Avora from reliably supporting 1,000–5,000 active students, what needs to be fixed, and what is the simplest architecture that gets us there?

Use the repository as the source of truth.

Audit first. Prove problems. Then remediate.

Heading